

單字練習台 v2 規格書

Flutter Web / Android / iOS · 學生與老師兩種角色 · 間隔重複、聽讀說寫、任務系統

版本：2026-09-11 規劃階段（開發前）

對象：開發者、老師、審閱者

配套文件：v2 Wireframes（21 頁，另冊）

現行網頁版（v1.7.2，學測衝刺版）：<https://joyce914apps.github.io/vocab-trainer/>

Repo：<https://github.com/Joyce914Apps/vocab-trainer>

目錄

1. 第一章 產品需求 (PRD)
2. 第二章 產品規格
3. 第三章 技術架構
4. 第四章 執行計畫
5. 第五章 決定紀錄
6. 第六章 AI 上稿模型評估
7. 第七章 第三方登入與服務申請
8. 附錄 v1.7 使用說明 (現行網頁版)

閱讀順序建議：先看第五章的決定紀錄了解已定案的事，再看第一、二章；技術與執行計畫給開發者；第六、七章是開工前的準備。教材品牌、作者與講義原始檔一律不出現在本書。

第一章 產品需求 (PRD)

版本：2026-09-10 修訂 2（加入角色與登入） | 狀態：規劃中，預計 W2-W4 課餘時間開工 | v1 (HTML) 已上線：<https://joyce914apps.github.io/vocab-trainer/>

1. 背景與問題 (Why)

v1 是為一位準備學測的高中生做的單一 HTML 工具，到 v1.7 已驗證的設計 (v2 全部承襲)：三段式練習 (預習→挖空回想→○△×)、前段連續的間隔階梯與七個可選方案、讀書日與前段不順延、拆字與同字根、四國口音內建音檔、文章先聽再看與逐字反白、逐句中譯、回音練習、每週講義的字組上課日與預習、找出生字的字典自動帶入。

但 v1 有四個結構性限制，改 HTML 解決不了：

1. **進度綁在一台裝置的一個瀏覽器**。換手機、換電腦、清瀏覽資料，進度就分家或消失。只能靠手動備份檔。
2. **只有一個學生**。第二位使用者要用，只能開另一個瀏覽器。
3. **教材寫死在程式裡**。新增一份講義要改 JavaScript 陣列。學測、英檢、TOEIC、國中小、自學教材沒辦法各自成套、各自選程度。
4. **排程邏輯沒有測試**。它是產品的核心，但目前只能靠人工試。

2. 目標與非目標

目標

- G0 兩種角色分流：學生（手機優先，練習）與老師（桌機優先，建班、上稿、發布、看進度）。**不強制登入** (App Store 5.1.1)：訪客可完整練習，加入班級、跨裝置同步、老師身分時才登入 (Google / LINE / Apple)，訪客進度併入帳號
- G1 同一個學生在手機、電腦、平板打開都是同一份進度，離線也能練，連線自動同步
- G2 學生可加入多個班級；每位學生在每套教材各有獨立進度
- G3 教材做成資料包，新增教材不改程式；每套教材有自己的考試類別與程度
- G4 排程引擎是純 Dart 套件，有完整單元測試，參數與理由寫在文件裡
- G5 v1 的備份檔可以直接匯入，舊使用者不掉進度
- G6 Flutter Web 先上線取代 v1；之後加 Android 只是多一個 build 目標

非目標 (v2.0 不做，v2.1 做)

- 老師自助上稿與 AI 產例句 / 改寫 / 題目 / 中譯 (v2.0 由開發者在同一介面代做，老師只審核與發布)
- LINE 推播提醒 (v2.1)

非目標 (v2 都不做)

- LINE / Liff 入口與推播 (見 ADR-0003，與 line_bento_order 重複，不做)
- 自動從 PDF 產生字卡 (轉檔工具只吃已整理好的八欄文字)

- 社群、排行榜、付費

3. 目標使用者與情境

使用者	情境	需要
準備學測 / 英檢的高中生（主要）	平日晚上 30—60 分鐘，週末只有零碎時間，常在手機上練	開了就能練、離線可用、進度不會丟、老師的講義自動出現
老師 / 家教 / 家長	每週一份講義，桌機上處理；想知道誰沒練、誰卡住	上稿快、發布後學生端自動排、儀表板一眼看完
準備 TOEIC / TOEFL / IELTS 的成人自學者	通勤時用手機，回家用電腦	跨裝置同步、能自己加教材
國中小學生	家長或家教陪同，用家裡共用的平板	同一裝置切換學生、介面字大
教材整理者（目前是開發者本人）	拿到講義後把單字整理成字卡	有明確的資料格式與轉檔工具

4. 使用者故事（核心）

- US1 身為學生，我在手機練完 20 個字，晚上打開電腦，複習行事曆顯示的是同一份進度。
- US2 身為家長，我在家裡平板上切換到弟弟的檔案，他的進度和姊姊的完全分開。
- US3 身為學生，我從「學測 L3—4」切換到「英檢中級」，兩套教材的到期複習分開計算，不會互相搶額度。
- US4 身為教材整理者，我把一份新講義轉成 JSON 放進 `data/courses/`，push 之後所有使用者下次開啟就看得見。
- US5 身為 v1 使用者，我上傳舊的備份檔，749 個字的熟悉度、複習日期、退步紀錄全部保留。
- US6 身為學生，沒有網路時我照樣練習，恢復連線後進度自動上傳，不需要按任何按鈕。
- US7 身為老師，我上傳講義後審核字卡、設上課日、發布，學生端隔天就排進去，不用我一個一個通知。
- US8 身為老師，我打開儀表板就知道誰三天沒練、誰在哪幾句卡住，可以在課堂上針對那幾句講。
- US9 身為家長，我替孩子建一個一人班，孩子用自己的 LINE 登入，我看得到他的進度但改不了他的練習。

5. 功能清單與優先級

優先	功能	說明
P0	排程引擎 (srs_engine)	間隔階梯、難度係數、同場回籠、出題模式判斷；純 Dart，100% 測試覆蓋
P0	教材包載入	讀 <code>courses.json</code> 目錄與各教材 JSON；程度與考試類別篩選
P0	今日練習	三段式流程、四種作答方式、○△× 標記、鍵盤快捷鍵
P0	本機儲存	Drift；Web 用 IndexedDB、原生用 SQLite；離線優先

優先	功能	說明
P0	首次使用引導	三張啟動卡（可跳過）、情境式 coach mark 各出現一次、老師端三步導覽
P0	直橫式與摺疊機	依視窗寬度重排且不丟狀態、避開鉸鏈、iPad 分割畫面
P0	訪客模式與登入	訪客可完整使用；Google / LINE / Apple 登入；登入時併入本機進度；學生 / 老師分流的路由與權限
P0	班級	老師建班、邀請碼、學生加入；講義發布到班級；Firestore 規則
P0	同步	登入即同步（原 P1 提前）：Firestore 離線快取、每張卡 LWW
P0	老師儀表板	班級總覽、每位學生、要注意清單；資料來自學生端既有欄位
P0	v1 備份匯入	讀 vocab-trainer-backup v1 格式
P0	Flutter Web 部署	GitHub Pages，取代 v1
P1	上稿介面	上傳 / 貼文字 → 抽字 → 審核 → 文章 → 產音檔 → 發布；v2.0 由開發者操作
P1	AI 上稿 (v2.1)	產例句、改寫文章、出題、逐句中譯，老師可改
P1	LINE 提醒 (v2.1)	老師從儀表板對學生發提醒
P1	字庫與複習行事曆	依熟悉度篩選、未來 30 天到期數
P1	設定	每日新字上限、複習上限、作答方式、語速、字體大小
P1	教材轉檔工具	tools/cards2json.dart：八欄文字 → 教材 JSON，含格式驗證
P1	教材包內容	拆字表、四國 mp3 與逐字時間點、逐句中譯隨教材包出貨
P1	講義流程	字組上課日、優先排程、上課前預習、過一遍
P1	任務系統	老師建範本、指派六種每日任務（課前聽、課後立即聽、跟述三部曲、單字、聽寫、課外閱讀）、本週表、錄音上傳、儀表板完成率
P1	逐句聽寫	文章逐句聽打、逐字比對、漏字清單
P2	文章閱讀	先聽再看、逐字反白、逐句中譯、隱性 wpm、題目、重讀排程
P2	語音	內建 mp3 播放 (just_audio)、回音練習、逐句跟讀、錄音對照 (record)、自己的音檔
P2	找出生字	內建字典、線上查、字庫字排入下一批
P2	列印今日	填空練習紙
P2	Android 版	同一套程式碼，加原生 Drift 初始化

6. 成功指標

- 排程引擎測試通過率 100%，且 v1 的排程行為在引擎測試中有對照案例（同輸入同輸出）
- v1 使用者匯入備份後，抽查 20 個字的 box / ease / due 與 v1 一致
- 手機瀏覽器首次載入 5 秒內可操作（4G），第二次 2 秒內
- 離線練習 50 題後恢復連線，30 秒內另一裝置看到更新
- 新增一套教材不需要改任何 Dart 程式碼

7. 風險與對策

風險	對策
搶走 stock_ledger 主線時間	只用 W2—W4 課餘；P0 做完就上線，P1/P2 排到 W8 之後
Flutter Web 首次載入慢	用 <code>--wasm</code> 或 CanvasKit 擇優；教材 JSON 延遲載入；預設字體不內嵌
未成年使用者的帳號與個資	用 LINE / Google 登入，不自管密碼；只存暱稱與進度；老師只能讀所屬班級學生的彙總與進度，不能讀備份內容
教材版權	老師上傳的原文與 PDF 存在班級私有空間，永不進公開資料與 repo；只有 AI 改寫版（老師勾選同意）可進公用教材
後端與 AI 費用	v2.0 後端只有 Auth、Firestore、LINE 驗證的 Function，免費額度內；v2.1 的 AI 按用量計費，一份講義數十元內，由老師端設定的金鑰或平台代付（待決）
同步衝突	進度以「卡片」為單位，取最後作答時間較新者（LWW），不做合併；作答紀錄 append-only
教材版權	例句改寫或自寫；雜誌文章一律原創改寫；教材包標明來源
v1 與 v2 並存期混亂	上線當天 v1 移到 legacy/，Pages 只留 v2；v1 的網址自動導向

8. 里程碑

里程碑	內容	預計
M1	packages/srs_engine 完成 + 測試 + 設計文件	W2
M2	登入與角色、班級、App 骨架、教材包載入、今日練習、本機儲存 + 同步、v1 匯入	W3
M3	Flutter Web 上線 GitHub Pages，取代 v1	W4
M4	老師儀表板、上稿介面（開發者操作）、字庫、行事曆、設定	W8 後
M5	文章、語音、Android / iOS 版	視 stock_ledger 進度
v2.1	老師自助上稿 + AI、LINE 提醒	M5 之後

對應 Notion 週次以 stock_ledger 主線為準，本專案不佔用主線任務時段。

第二章 產品規格

本文件定義畫面、流程、規則與資料格式。排程規則的數值全部承襲 v1，理由見 01_PRD.md 與 README「設計決策」。

0. 角色與權限

角色	入口	能做	不能做
學生	手機優先，底部 5 分頁	練習、看文章、加入班級、選公用教材、備份	看別人的進度、改講義
老師 / 家長	桌機優先，左側 4 項（儀表板、講義、班級、教材庫）	建班、邀請、上稿、發布、設上課日、看所屬班級學生的彙總與進度、指派公用教材	讀學生備份內容、改學生的練習設定
開發者	老師介面 + 教材庫管理	維護公用教材、v2.0 代老師上稿	

登入：**不強制**（App Store 5.1.1）。啟動直接進訪客的「今日」，資料存本機；加入班級、跨裝置同步、老師身分時才登入（Google / LINE / Apple，iOS 必備 Apple）。登入時把訪客的本機進度併入帳號（每張卡取 updatedAt 較新者）。首次登入選角色；同一帳號可兼老師與家長身分。訪客的「加入班級 / 登入同步」提示只在前三天顯示於今日卡片，之後收進設定。

班級模型：`teachers/{uid}` → `classes/{cid}`（邀請碼、預設讀書日與方案）→ `members`；`lessons/{lid}`（屬於老師，發布到一或多個班級，含字組、文章、音檔路徑、上課日）；學生進度在 `users/{uid}/...`，老師對所屬班級學生有讀權限。原文 PDF 存 `classes/{cid}/private/`，永不公開。

0a. 首次使用引導（v2.0）

原則：**不擋路、可跳過、只出現一次、在需要的當下出現**。分三層：

- 啟動引導（三張卡，可跳過）**：第一次打開 App 顯示。①「三段式：先認識、再回想、最後自己標 ○△×」②「每天固定量，前幾天連續出現是設計不是重複」③「讀書日與每日新字」（直接在這張卡上設定，預設二三四五、8 字）。最後一張有「有班級代碼？」與「先自己練」兩顆按鈕。
- 情境提示（coach mark，各只出現一次）**：第一張預習卡（手打一次的用意）、第一次對答案（○△×可以自己改、「矇到的請改 △」）、第一次答錯回籠（「不是重複，是剛忘掉的要馬上補」）、第一篇文章（先聽不看文字的理由）、第一次回音練習（停下來聽回音）、第一次休息日（為什麼只有幾個字）、第一次講義上課日。每個提示一句話加「知道了」，可在設定「重看所有提示」。
- 老師端引導**：第一次進老師首頁：建班 → 取得邀請碼 → 套用任務表範本，三步導覽，可跳過。

記錄：`onboarding.done`、`coachmarks.seen[]` 存在帳號（訪客存本機，登入時併入）。v1 備份匯入提示放在啟動引導最後一張。

1. 資訊架構

啟動 → 今日（訪客，直接可練）

└─ 需要時登入（加入班級 / 設定的帳號區 / 我是老師）

└─ 學生：加入班級（代碼）→ 講義自動出現 → 主畫面（底部 5 個分頁）

└─ 老師：建立班級 → 老師首頁（儀表板 / 講義 / 班級 / 教材庫）

學生主畫面

└─ 今日 三段式練習

└─ 字庫 依熟悉度 / 字組篩選，點開看卡片

└─ 文章 閱讀 + 跟讀（P2）

└─ 行事曆 未來 30 天到期數、逾期

└─ 設定 教材選擇、額度、作答方式、聲音、帳號與同步、備份、重置

教材切換放在「設定」而不是主畫面，因為一位學生大多數時候只練一套；切換是低頻操作。

2. 核心流程：今日練習

buildQueue()

到期複習（ $\text{due} \leq \text{今天}$ ，依退步次數多→少、到期早→晚排序，取前 revCap ）

+ 新字（未學過，依教材順序，取前 goal ）

→ 逐題：

未學過的字：預習（顯示例句、音節拆解、手打一次）→ 回想 → 標記

學過的字：直接回想 → 標記

→ 標記後：

○ $\text{box} + 1$ 、 $\text{ease} + 0.10$ （上限 2.9）

△ $\text{box} - 2$ （下限 0）、 $\text{ease} - 0.15$ （下限 1.3）、 $\text{reps} > 1$ 則 $\text{lapses} + 1$

× $\text{box} = 0$ 、 $\text{ease} - 0.30$ （下限 1.3）、 $\text{reps} > 1$ 則 $\text{lapses} + 1$

$\text{due} = \text{snap}(\text{今天} + \max(1, \text{round}(\text{LADDER}[\text{box}] \times \text{ease} / 2.5)))$

snap ：若落在休息日，順延到下一個讀書日（最多往後找 14 天）

同場回籠：非 ○ 且該字本場回籠 < 2 次 → × 插到 4 題後、△ 插到隊尾

排程方案（v1.2 起，v2 承襲）

id	名稱	間隔（天）	自適應	跨度	適合
mix	考前混合（預設）	1,1,1,1,7,15,30,60	是	116	學測、四個月以上
f16	固定 16 遍	1,1,1,1,2,2,3,3,7,7,7,7,10,10	否	70	起步者、國中小
f10	固定 10 遍	1,1,1,1,7,7,7,30,30	否	86	英檢初級、多益 600
f8	固定 6—8 遍	1,1,1,1,7,7,30,30	否	78	英檢中級、多益 700
e5	經典 5 遍	1,2,3,8,15	否	29	一個月一輪、每天固定量（艾賓浩斯經典表：第 2、4、7、15、30 天）
s8	衝刺 8 週	1,1,1,1,7,7,15	是	33	剩兩到三個月
s4	衝刺 4 週	1,1,1,1,7	是	11	剩一個月內

- box 到達 $\text{ladder.length} - 1$ 視為畢業，之後以最後一格の間隔持續複習

- 前段長度 `earlyLen` = 階梯開頭連續 1 的個數 (4 或 5)
- 自適應方案：○ `box+1`、`ease+0.10`；△ `box-2`、`ease-0.15`；× `box=0`、`ease-0.30`；間隔 = `ladder[box] × ease / 2.5`
- 固定遍數方案：○ `box+1`；△ `box` 不變 (同格再走一次)；× `box-1`；不用 `ease`
- 切換方案時保留每個字的 `box`，之後的間隔改查新階梯
- 目標日建議：跨度 $\leq$ 剩餘天數 $\times$ 0.9 的最長方案；沒有則衝刺 4 週

同場再看

- 答錯 (△×)：既有規則，× 插到 4 題後、△ 插到隊尾，每字每場最多兩次
- 答對的新字 (第一次作答)：排到隊尾再看一次；第二次答對只記錄，不推進 `box`、不改到期日；第二次答錯照一般規則處理但不計退步

讀書日與前段

- `strictEarly` (預設開)：`box < earlyLen` 的字到期日照日曆天，不順延；休息日打開 App 只出這些字，並顯示預估分鐘數
- 關閉時：所有到期日都順延到下一個讀書日，休息日不排除任何進度

拆字與同字根 (v1.3)

- MORPH 表：`word` → '`pre-|前;cept=cap|抓;-ion|名詞`'。form 結尾 - 為字首、開頭 - 為字尾、其餘為字根；`form=key` 把不同拼法歸到同一字根群。人工整理，只收有把握的分析
- 預習卡與對答案卡：音節拆解下方顯示「拆字」與「同字根」(字庫裡同鍵的字，最多 8 個，標未學)。回想卡不顯示，避免洩漏字義
- 字庫篩選「依字根 / 字尾分組」：字根群 (≥ 2 字) → 字首 → 字尾，每群一個可展開列
- `groupRoots` (預設開)：挑新字時，同字根的未學字拉到相鄰位置，不超過每日額度
- v2 教材包：拆字資料放在卡片的 `morph` 欄位，同一格式

音檔 (v1.4)

- 內建：`audio/w/<slug>_<accent>.mp3` (單字)、`audio/s/<slug>_<accent>.mp3` (例句)、`audio/a/<artId>_<accent>.mp3` (整篇)、`audio/a/<artId>_s<n>_<accent>.mp3` (逐句，分句規則與 App 相同)。accent ∈ en-US / en-GB / en-AU / en-CA
- 產生：`tools/gen_audio.py` (神經網路語音，四國口音各一個女聲)，ffmpeg 壓成 32 kbps 單聲道 (文章 48 kbps)
- 播放：`sayAudio(kind,key,txt)` 先放 mp3，`onerror` (404、離線) 退回 `speak()` 瀏覽器語音；速度跟設定的語速
- 文章流程 (v1.4.1)：打開文章先只有播放器與「第一遍，先聽不看文字」提示，內文隱藏；音檔播到底 `onended` 才啟用「看文章對照」並累計 `read[id].listens`；可「直接看文章」跳過。看文章後才有再聽一次、逐句跟讀、理解題。沒有計時按鈕：`reveal()` 記 `rd.t0`，`finishArt()` 算 wpm (含點中譯與跟讀時間，第一次偏低是基準線)，仍記最佳與重讀排行
- 逐字反白 (v1.5.1)：`gen_audio.py` 用 `boundary='WordBoundary'` 串流取得每字 `offset / duration`，存 `<id>_<accent>.words.json` ([起秒, 迄秒, 文字])；App 的 `bodyHTML` 把每個字包成 `.tw[data-i]`

[data-s]，alignWords() 依序正規化配對（實測 100% 對齊），karaTick() 用 requestAnimationFrame 二分搜尋當前字，加 .lit，同句加 .lit-s，超出視窗則 scrollIntoView。自己上傳的音檔沒有時間點，不反白

- 逐句中譯 (v1.5.2)：ARTZH[artId][k] 人工翻譯，句序與 rd.sens 一致（建檔時 assert）。bodyHTML 每句包 .sen[data-s]，點句展開 .senz (一次一句；點 .kw 不觸發)，記錄 read[id].zhOpened[k]。「全部中譯」在 finishArt 後才啟用。v2 教材包：文章的 sentences_zh 陣列
- 文章頁：<audio controls> 播放器；「用自己的音檔」把檔案存進 IndexedDB (vocab_audio/own，key = 文章 id，上限 60 MB)，優先於內建；可移除
- v2：音檔路徑寫進教材包 (audio 欄位或約定命名)，Flutter 用 just_audio 播放，PWA 快取常用音檔

每週講義與字組上課日 (v1.6)

- 貼上字卡：deck = 來源名稱（原本全部進「匯入字卡」），字組開關自動出現
- S.deckDue[deck] = 'yyyy-mm-dd'；deckDue(n) 只在日期 ≥ 今天時有效
- buildQueue 挑新字：先依 deckDue 由近到遠，再 pin，再字組原順序；仍受每日額度限制
- 字組列顯示「上課日・還有 N 字，M 個讀書日，每天約 X 個」，X > goal 時警示；今日卡片列出所有有效上課日的字組
- 「過一遍」：字庫篩選 deck:<name>，依字組原順序列出，不排序、不影響排程
- 預習：LESSON_ARTS[deck] 對應文章；previewItems() 算預習日 = 上課日前一天，往前找最近的讀書日（已過則今天）。今日卡片與行事曆在該日多一項「上課前預習」，不影響 artPlan 的一般輪序與重讀
- 不做「只練這組」：會暫停其他字組的複習
- v2：教材包的 deck 加 lesson_date，profile 層可覆寫

找出生字 (v1.5)

- 比對順序：字庫 (含 base() 變化形) → data/dict.json (w 字→[中文, 詞性]，l 變化形→原形) → 空白 (可線上查)
- 字庫已有：顯示熟悉度標籤；未學者可勾選，pin=true；buildQueue 挑新字時 pin 者優先，仍受每日額度限制
- 新字：建卡時 deck='我的文章'、kind='r'、pin=true、例句為文章原句、exz 選填
- 線上查：免費翻譯 API (langpair=en|zh-TW)，回傳含 QUERY LENGTH / LIMIT 視為失敗
- 字典來源：開源英漢字典 (MIT，見 THIRD_PARTY_NOTICES.md) 取有考試標籤或語料庫前 15,000 的純小寫字，簡轉繁，每字最多兩個詞性、各三個義項

回音練習 (v1.7)

- 需要內建音檔的逐字時間點；echoChunks() 依 .tw 順序切段：滿 6 字、或 ≥4 字遇標點、或句尾就斷，<3 字的尾巴併入前一段 (上限 8 字)
- 每段四步：echoPlay 用 currentTime 播 s-0.06 到 e+0.12 → 停 1.5 秒 (聽回音) → 模仿 (max(1.5s, 字數×0.52s)) → 再播；三輪後停下等使用者按「再一次」或「下一段」

- 面板顯示段數、輪數、計時；十分鐘自動結束；`read[id].echoAt` 記錄段數、`echoMin` 累計分鐘
- 內文加 `.echo` 時其他字淡出，當前段 `.ech` 反白
- 單字跟讀與逐句跟讀改為四步：聽 → 停 1.5 秒 → 唸 → 再聽

任務系統（老師指派，v2.0）

把老師的紙本「每周在家學習進度表」數位化：老師建立一週任務表 → 學生每天做完自動打勾填日期 → 老師看到同一張表。一列一種任務，一格一天，休息日打 X；「不可跳格」由系統保證：前一天沒做的項目以「補做」顯示。

六種任務類型：

類型	老師設定	學生動作	完成判定	證據
課前聽	單元（預設下一堂講義）、每天幾軌（2）、開始日（上課前 8 週）、目標遍數（5）	整軌播完，可邊聽邊跟	每軌播到底計一次	遍數
課後立即聽	單元（正在上的）	上課日當天播一遍	播到底	系統紀錄
跟述三部曲	單元、每天幾軌（2）、是否錄音	① 只聽 ② 回音練習（看文字） ③ 不看文字跟述並錄音	三步完成	錄音檔
單字	每日新字、複習上限（沿用排程）	現有練習	業績達成	○△×
聽寫	軌（老師挑或系統建議）、次數上限（3）	逐句聽、打整句、逐字比對	全部句子送出	正確率、漏字清單
課外閱讀	每天頁數（10 / 20）、是否朗讀錄音	填書名、頁數、上傳一頁朗讀；不查字典	頁數達標 + 錄音	錄音、書名

| 外部英聽 | 每週幾集、素材清單（老師在班級設定維護：節目名或連結） | 填來源與集數；勾 ① 不看字幕聽大意 ② 開字幕找沒聽出來的字 ③ 口說摘要三句並錄音（可略） | ①② 勾選；③ 錄音 | 錄音、來源 |

另有「自由任務」：老師輸入文字，學生勾選，可拍照。外部英聽的素材由 App 外的平台播放，App 只記錄與收錄音，不代管內容。

- 單元 = 講義（lesson），軌 = 文章（article）。系統依上課日自動分「已上過 / 正在上 / 下一堂」，課前聽指向下一堂，聽寫與跟述只能挑已上過的
- 老師端：任務表範本（建一次，每堂課套用到新的一週）、指派到班級或個別學生並可調量、本週表（同紙本格子，點格看證據）、儀表板加本週完成率與聽寫正確率
- 學生端：今日卡片改成任務清單（每項有進度與「去做」）、本週表、錄音上傳
- 任務系統不改排程引擎，是架在既有功能上的清單與勾選層

逐句聽寫（v2.0，任務系統的一種）

- 用逐句 mp3：播一句（最多 3 次，可慢速）→ 打整句 → 逐字比對（正規化大小寫與標點）→ 標出漏字、錯字、多字
- 一篇做完顯示正確率與漏字清單；漏字若是字庫字，可一鍵排入下一批；漏掉的句子可直接跳到回音練習

- 記錄 `read[id].dictation = {date, acc, missed[]}`

每日業績

- `doneDays`：有作答的日期集合，起始日 = 最早的一天
- 今日卡片顯示：第 N 個讀書日、連續天數（休息日不斷鏈）、新字與複習各完成幾個、達成打勾
- 行事曆摘要顯示累計讀書日與連續天數

讀書日

每位學生設定一週哪幾天讀書（預設二三四五，至少一天）。所有到期日（單字、文章重讀）都經過 `snap` 順延；休息日打開 App 顯示「今天是休息日」而不排進度。行事曆的休息日列淡化顯示，新字預估只算在讀書日。每週實際新字量 = $\text{goal} \times \text{讀書日天數}$ ，UI 要明示這個數字。

作答方式

模式	規則
auto（預設）	$\text{kind} = r \rightarrow \text{choice}$ ； $\text{lapses} \geq 2 \rightarrow \text{spell}$ ； $\text{box} \geq \text{earlyLen}$ 且 $\text{reps} \% 3 == 0 \rightarrow \text{spell}$ ；其餘 choice
choice	挖空例句 + 四選一。干擾項優先同教材同詞性，其次同教材，其次首字母相同且長度相近
spell	手打完整單字，Levenshtein 距離 ≤ 1 視為 Δ 候選
dictate	聽寫打字：只播例句，不給中文與挖空句，打出目標字；判定同 spell。由設定 <code>spellByEar</code> 把 spell 轉成 dictate，不是獨立模式

學習計畫試算（設定頁）

改任一輸入（每日新字、讀書日、方案、目標日、字組開關）即時更新：起始日與累計讀書日、字庫（未學 / 學習中 / 畢業）、每週新字、新字學完日（依讀書日逐日數）、全部畢業預估（學完日 + 方案跨度，順延到讀書日）、每日負擔（新字 + 未來兩週讀書日平均複習）、一個字的旅程（由階梯產生的文字）、目標反推（每日至少幾個、建議方案、一鍵套用）。

3. 帳號與學生

- 一個登入帳號 = 一位學生（或一位老師）。家長替孩子用：家長建一人班，孩子用自己的 LINE / Google 登入
- **進度以「字」為單位、跨教材共用**：同一個字（以原形 lemma 為鍵）不論出現在學測、英檢講義或自己貼的卡，都只有一筆進度，不會因為同時開兩套教材而複習兩次。教材只是「引用」這個字並附自己的例句；複習時顯示最近學過的那套教材的例句，其餘例句在字卡可切換
- 老師上稿時系統標出「這 N 個字已在公用教材或其他講義」，不重複建卡，但仍列入該講義的「過一遍」清單與上課日優先（未學者）
- **班級講義與公用教材可同時開**（已定），每日額度共用，今日清單先講義後公用
- 登出保留本機資料；換帳號登入會另建本機資料
- 刪除帳號需二次確認，且先強制下載備份

3a. 老師端

- **儀表板**：本週卡（學生數、有練人數、平均連續）、講義卡（完成率、文章預習）、要注意（3 天沒練、本週退步 ≥ 10 、講義每日需求超過上限）；學生表（連續、本週新字 / 複習、講義進度、文章、回音分鐘、最近）；點學生看細節（熟悉度分布、中譯點最多的句子、退步最多的字、**每個字的** ○△×）
- **每個字的** ○△×（**推導值**）：不是使用者最後一次點的標記，而是由累積紀錄算出的狀態，規則：
 - × 生疏： $\text{lapses} \geq 2$ ，或最近 5 次作答有 2 次以上 ×，或 $\text{box} = 0$ 且 $\text{reps} \geq 2$
 - △ 卡點：不符 × 且 ($\text{box} < \text{earlyLen}$ ，或最近 5 次有 △ / ×，或 $\text{lapses} = 1$)
 - ○ 穩定： $\text{box} \geq \text{earlyLen}$ 且最近 5 次全 ○ 且 $\text{lapses} = 0$
- 未學： $\text{seen} = \text{false}$
- 儀表板可依這個狀態篩選（例如列出某講義所有 × 的字），也是「退步最多」的來源。引擎提供 `derivedMark(progress, recentLog)` 純函式，App 與儀表板共用
- **上稿（三步）**：① 上傳 PDF / 貼文字（掃描檔提示改貼文字）② 審核字卡：抽字（字庫比對 + 字典），勾選、改中文、P / R、例句（v2.1 可 AI 產）③ 文章（貼原文僅班級可見，或 AI 改寫）、題目、逐句中譯、產音檔（伺服器，四國口音 + 逐字時間點）、設上課日、發布到班級
- **班級**：改名、邀請碼、LINE 分享連結、預設讀書日 / 每日新字 / 方案、成員名單與移出
- **教材庫**：我的講義（草稿 / 已發布）、公用教材（指派給班級）
- 發布後可修改，學生端下次啟動更新，已學進度不受影響

4. 教材包 (Course Pack)

目錄 `data/courses.json`

```
{
  "version": 1,
  "courses": [
    { "id": "gsat",      "name": "學測",      "exam": "gsat", "level": "高中",  "file":
"courses/gsat.json",    "words": 749 },
    { "id": "gept-int", "name": "英檢中級", "exam": "gept", "level": "中級",  "file":
"courses/gept-int.json", "words": 0 },
    { "id": "self-study", "name": "自學教材",  "exam": "other", "level": "自學",  "file":
"courses/self-study.json", "words": 0 }
  ]
}
```

exam 列舉：`gsat`、`gept`、`toEIC`、`toefl`、`ielts`、`elementary`、`junior`、`other`。UI 依 exam 分群、依 level 排序。

教材 `data/courses/<id>.json`

```
{
  "id": "gsat",
  "name": "學測",
  "version": "2026-09-09",
  "decks": [
    {
      "id": "l34",
      "name": "學測 L3-4 高頻字",
      "defaultKind": "p",
      "cards": [
        {
          "id": "b0",
          "en": "abandon", "pos": "v.", "zh": "放棄；拋棄",
          "ex": "They abandoned the plan.", "exz": "他們放棄了那個計畫。",
          "form": "abandoned", "note": "", "syl": "aban·don", "fam": ["abandonment"],
"kind": "p",
          "morph": "ab-|離開;band|禁令"
        }
      ]
    }
  ],
  "articles": [ { "id": "a1", "title": "...", "level": "英檢中級", "tag": "...", "body": "...", "questions": [] } ],
  "sources": [ { "id": "s0", "name": "...", "file": "...", "date": "2026-08-19", "note": "..."} ]
}
```

- `id` 在教材內唯一且永不改變，進度靠它對應。gsat 教材沿用 v1 的 `b<i>`、`d<deck>_<i>` 編號，讓 v1 備份可直接對應
- `kind`：p 要會拼寫、r 看得懂就好；卡片沒填則用 deck 的 `defaultKind`
- 卡片欄位除 `id`、`en`、`zh` 外皆可省略

八欄文字格式（給 AI 與人工整理用，與 v1 相同）

單字 | 中文 | 詞性 | 英文例句 | 例句中譯 | 用法筆記 | 音節拆解 | P或R

`tools/cards2json.dart` 負責轉成上面的 JSON，並檢查：單字不重複、id 不與既有衝突、例句包含該字或其變化形。

5. 進度資料

每（帳號 × 字）一筆，鍵為 lemma；教材與卡片透過 lemma 對應到同一筆：

```
cardId, box, ease, reps, lapses, streak, seen, due(yyyy-mm-dd), last(yyyy-mm-dd),
kindOverride(p/r/null), updatedAt(ISO, 同步用)
```

作答紀錄另存 review_log (append-only) : cardId, date, mark(o/t/x), mode, elapsedMs 。v1 只留最近 14 筆，v2 全留，行事曆與統計靠它。

自訂字卡（使用者在裝置上貼的）存在檔案層，內容與進度都在本機 / 雲端，不進教材包。

6. 備份與匯入

- 匯出：{ format: "vocab-trainer-backup", version: 2, profile, courses: { <courseId>: { progress, customCards, log } }, settings }
- 匯入 v1：讀 version: 1 的 state.recs，id 以 b、d 開頭者對應 gsat 教材；以 u 開頭者轉成自訂字卡；state.sources 中 own: true 者一併帶入；goal、revCap、mode、rate 進設定
- 匯入採合併而非覆蓋：同一張卡取 updatedAt (v1 用 last) 較新者

7. 同步 (P0)

- 登入方式：Google、LINE (Cloud Function 驗證 LINE access token 後簽發 Firebase custom token)、Apple
- Firestore 結構：users/{uid}/words/{lemma} (跨教材共用的進度)，review_log 為子集合；教材卡片以 lemma 對應；classes/{cid}、lessons/{lid} 見第 0 節
- 離線優先：本機 Drift 為真相，Firestore 開離線快取；每筆寫入帶 updatedAt，衝突取新者
- 安全規則：users/{uid} 本人可寫；所屬班級的老師可讀 progress 彙總；classes/{cid} 老師可寫、成員可讀；lessons/{lid} 老師可寫、發布班級的成員可讀；classes/{cid}/private 只有老師

8. 設定項

項目	預設	範圍
每日新字 goal	8	3–60 任意整數
讀書日 days	二三四五	一週 1–7 天
排程方案 plan	mix	六選一
目標完成日 targetDate	空	全域一個 (例如考試日)；空 = 「程度提升」模式，試算不做反推與建議方案
新字前段不順延 strictEarly	開	
答對新字當場再看 sameDay	開	
同字根新字連著背 groupRoots	開	
拼寫題用聲音出題 spellByEar	關	開 = spell 改為 dictate
每日複習上限 revCap	60	10–200
作答方式 mode	auto	auto / choice / spell (recog、listen 已移除，舊值轉 auto)
語速 rate	0.85	0.6 / 0.85 / 1.0

項目	預設	範圍
進新字自動朗讀	開	
文章間隔 artEvery	3 天	1–14
字體大小	17	14–22
主題	跟隨系統	亮 / 暗 / 系統

9a. 通知

- FCM (Android / iOS) + Web Push (PWA, VAPID)。預設開，設定可關
- 內容由 Cloud Function 在每個讀書日的固定時間（預設 19:00，可改）依到期數送：「今天有 N 個字到期，新字 M 個」；講義上課前一天加一句「明天上課，先聽兩篇文章」
- 老師端 v2.1 可對學生發 LINE 提醒 (Messaging API)，與推播分開

9b. 行事曆

- 三個尺寸一律月曆格；點某天顯示當日明細（手機以底部抽屜）。視覺稿由 Claude Design 產出，不在規格內

9c. 教材更新

- App 啟動時讀伺服器的 `courses.json` 版本與班級 `lessons` 的 `updatedAt`，比本機新就背景下載並提示「教材已更新」；已學進度不受影響

9d. 直式、橫式與摺疊機

- 直式為主，橫式都要能用。手機橫式：練習卡改左右兩欄（左：字與例句，右：作答區與 ○△×），文章頁左內文右播放器與回音面板，行事曆改成月曆格橫排；不鎖定方向
- 平板：直橫式都支援；iPad 分割畫面與 Slide Over 依實際寬度套用手機或平板版型
- 摺疊機 (Android 摺疊、雙螢幕)：
 - 依「目前視窗寬度」而不是裝置型號決定版型；展開 / 收摺時視窗大小改變，畫面即時重排，**不重建頁面、不丟狀態**（練習中的那張卡、播放進度、回音段數都保留）
 - 封面小螢幕（窄長）用手機版型；內螢幕（接近正方形、寬 ≥ 600）用平板版型的兩欄
 - 避開鉸鏈：用 `MediaQuery.displayFeatures` 取得鉸鏈位置，兩欄版型以鉸鏈為分界，文字與按鈕不落在鉸鏈上；半開（桌面模式）時上半內容、下半控制（播放器、作答區）
 - 練習卡最寬 640px 置中，避免在超寬螢幕上一行太長
 - 鍵盤與外接：平板外接鍵盤時 Enter / 1–4 快捷鍵同桌機；滑鼠 hover 只做輔助，不承載功能
 - 測試矩陣：手機直 / 橫、平板直 / 橫、iPad 分割畫面 1/3 與 1/2、摺疊機封面 / 內螢幕 / 半開、桌機 1024 與 1440

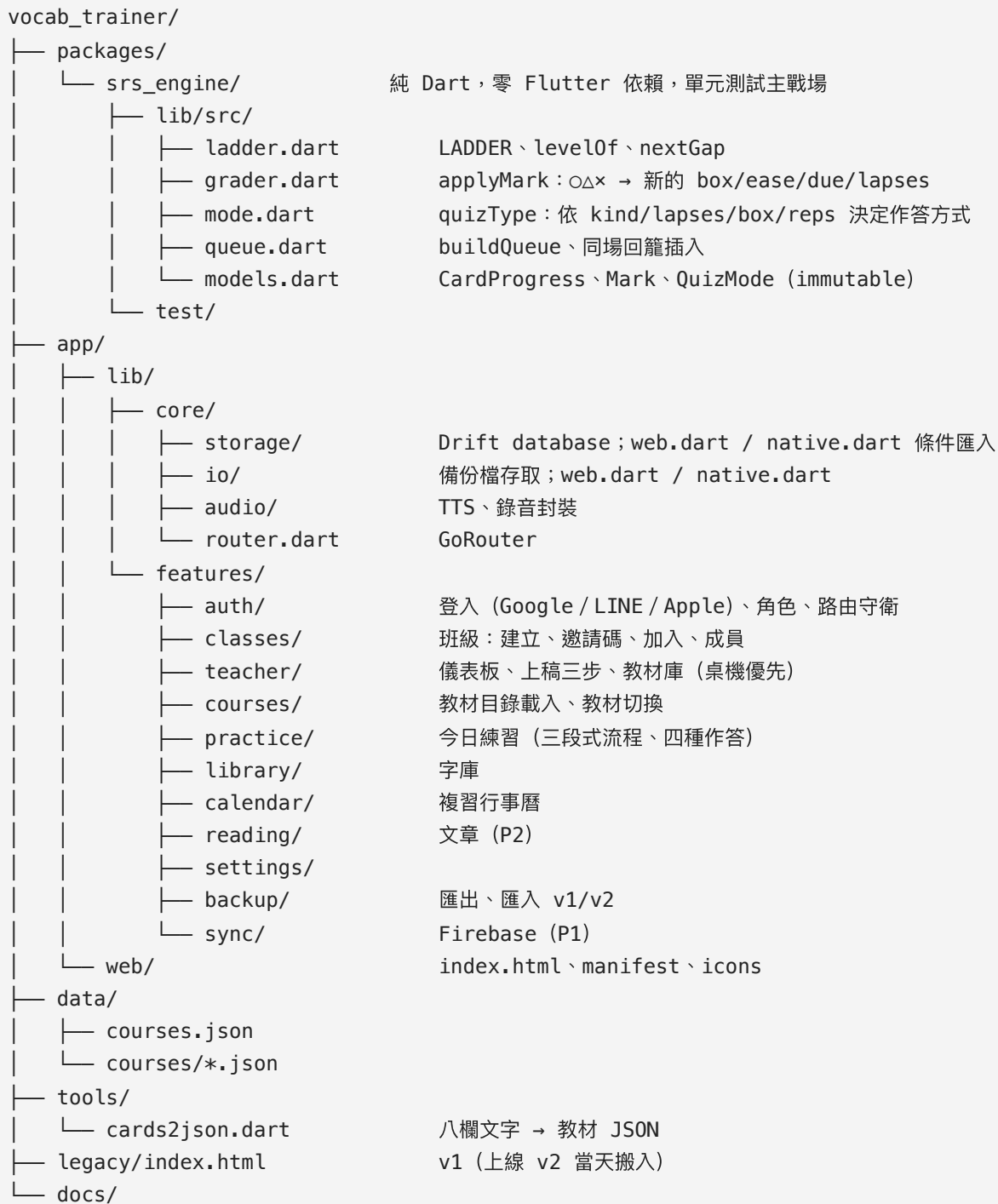
9. 無障礙與裝置

- 最小可用寬度 360px；選項按鈕高度 $\geq 44\text{px}$
- 鍵盤：Enter 送出 / 下一步，1-4 選項，○△× 對應 1 2 3
- 深色模式跟隨系統
- 所有文字可放大到 22px 不破版

第三章 技術架構

1. 架構總覽

結構比照 `stock_ledger`：純 Dart 引擎套件與 Flutter App 分離，App 內 Clean Architecture + Feature-First。



依賴方向：presentation → domain ← data，app → srs_engine。引擎不知道 Drift、Firebase、Flutter 的存在。

狀態管理 Riverpod 3（原生寫法，不用 codegen，與 line_bento_order 一致）。路由 GoRouter。

2. srs_engine API

```

/// 排程方案：間隔階梯 + 是否自適應（七個內建：mix / f16 / f10 / f8 / e5 / s8 / s4，見產品規格書）
@immutable
class SchedulePlan { final String id; final String name; final List<int> ladder; final
bool adaptive; }
const plans = <String, SchedulePlan>{ ... };
/// 剩 remainingDays 天時建議的方案：跨度  $\leq 0.9 \times \text{remainingDays}$  的最長者
SchedulePlan suggestPlan(int remainingDays);

enum Mark { ok, hard, miss }           // ○ △ ×
enum QuizMode { auto, choice, spell, recog }
enum CardKind { production, recognition } // p / r

@immutable
class CardProgress {
  final String cardId;
  final int box; final double ease; final int reps; final int lapses; final int streak;
  final bool seen; final DateTime? due; final DateTime? last;
  final CardKind kind;
  const CardProgress.fresh(String cardId, {CardKind kind = CardKind.production});
}

/// 目前熟悉度階層 0–8 (0 未學、8 畢業)
int levelOf(CardProgress p);

/// 下次間隔天數： $\max(1, \text{round}(\text{ladder}[\text{box}] * \text{ease} / 2.5))$ 
int nextGap(CardProgress p);

/// 讀書日順延：若 date 不在 studyDays (0=日...6=六)，往後找最近的讀書日（最多 14 天）
DateTime snapToStudyDay(DateTime date, Set<int> studyDays);

/// 套用一次作答，回傳新的進度（純函式，today 由外部注入以便測試）
/// isAgain：同場再看的第二次作答（答對不推進、答錯不計退步）
/// strictEarly：前段 (box < earlyLen) 照日曆天，不順延到讀書日
CardProgress applyMark(CardProgress p, Mark m, DateTime today,
  {required SchedulePlan plan, required Set<int> studyDays, bool isAgain = false, bool
strictEarly = true});

/// 依設定與進度決定作答方式
QuizMode quizType(CardProgress p, QuizMode setting);

/// 由累積紀錄推導的狀態（未學 / ○穩定 / △卡點 / ×生疏），儀表板與字庫共用；規則見產品規格書 §3a
DerivedMark derivedMark(CardProgress p, List<ReviewLogEntry> recent);

/// 建立今日隊列：到期複習（排序、上限）+ 新字（上限）
List<String> buildQueue({
  required Iterable<CardProgress> all,
  required DateTime today,
  required int goal, required int revCap,
});

```

```
/// 同場回籠：回傳插入位置（null 表示不回籠）
int? requeueIndex({required Mark m, required int idx, required int queueLength, required
int timesRequeued});
```

所有函式純粹、無 IO、無隨機（干擾項與洗牌在 App 層，引擎只給規則），因此測試不需 mock。

引擎以外、v1 已有而 v2 要搬的純邏輯

模組	內容	v1 位置
morph	拆字表解析、同字根查詢、分組	morphOf / rootKeys / siblings / morphGroupsHTML
lesson	字組上課日、優先排序、預習日（上課前一天往前找讀書日）	deckDue / deckPlanLine / previewItems
echo	依逐字時間點切 4-6 字片段、四步狀態機、十分鐘計時	echoChunks / echoRun
karaoke	時間點對齊內文、二分搜尋當前字	alignWords / karaTick
scan	字庫比對（含變化形）、字典查詢、線上查	libFind / bigLookup / lookupOnline

這些都是純函式或狀態機，放在 `packages/srs_engine` 或 `packages/content_tools`，一樣先寫測試。

3. 資料模型（Drift）

表	主鍵	欄位摘要
profiles	id	name, createdAt, lastUsedAt, cloudUid?
course_progress	(profileId, courseId)	activeDeckIds(json), lastPracticedAt
word_progress	(uid, lemma)	box, ease, reps, lapses, streak, seen, due, last, kindOverride?, lastCourseId, updatedAt。跨教材共用：同一個字在多套教材出現只有一筆
course_cards	(courseId, cardId) → lemma	教材卡片對進度的對應；同 lemma 在不同教材各有例句
review_log	autoId	profileId, courseId, cardId, date, mark, mode, elapsedMs
custom_cards	(profileId, id)	courseId?, en, zh, pos, ex, exz, note, syl, kind, sourceId, updatedAt
custom_sources	(profileId, id)	name, file, date, note
settings	profileId	goal, revCap, mode, rate, autoSay, artEvery, fontSize, theme, studyDays(json), planId, targetDate, strictEarly, sameDay
daily_stats	(profileId, date)	newDone, reviewDone（每日業績與連續天數由此算）
article_progress	(profileId, articleId)	reads(json: date, wpm, score), nextDue

教材內容不進資料庫，每次啟動從 `data/courses/*.json` 讀（Web 用 `fetch`、原生用 `assets`），以 `courseId + cardId` 與進度 join。

平台分岔：

```
// core/storage/db.dart
import 'db_native.dart' if (dart.library.js_interop) 'db_web.dart';
```

Web 用 `drift/wasm.dart` (需 `sqlite3.wasm` 與 `drift_worker.js` 放進 `web/`)，原生用 `drift/native.dart`。

4. 教材載入

<code>CourseCatalogRepository.load()</code> 並通知	→ 讀本機 <code>courses.json</code> ；啟動時比對伺服器版本，較新則背景更新
<code>CourseRepository.load(courseId)</code> <code>articles, sources)</code>	→ 讀單一教材 JSON，解析成 <code>Course(decks, cards,</code>
<code>ActiveCourse (Riverpod)</code>	→ 目前 <code>profile</code> 選定的教材
<code>CardWithProgress</code>	→ <code>Course.card × card_progress, practice</code> 用這個

教材 JSON 的解析用 `json_serializable` 或手寫 `fromJson`，以 `tools/validate_course.dart` 在 CI 驗證格式。

5. 登入、班級與同步 (P0)

- `firebase_auth`：Google (`google_sign_in`)、Apple (`sign_in_with_apple`)、LINE (`flutter_line_sdk` 取得 access token → Cloud Function `lineAuth` 驗證後 `createCustomToken`)。Web 端 LINE / Apple 走網頁授權
- 訪客：未登入時 Drift 以本機 guest id 存資料；登入後 `mergeGuestInto(uid)` 逐卡比較 `updatedAt` 併入，再清掉 guest 資料
- 角色存 `users/{uid}.role` (student / teacher)，路由守衛：未登入一律 `/s/*` (訪客)，老師功能需登入且 `role=teacher`
- 班級：`classes/{cid}` (`ownerUid`、`name`、`code`、`defaults`)、`classes/{cid}/members/{uid}`；加入用邀請碼經 Function `joinClass` 寫入 (避免學生自行寫任意班級)
- 講義：`lessons/{lid}` (`ownerUid`、`classIds[]`、`deck`、`articles`、`audio` 路徑、`lessonDate`、`status`)；音檔放 Storage `lessons/{lid}/audio/`；原文 PDF 放 `classes/{cid}/private/` (規則只有 owner)
- 上稿 Functions (v2.1)：`extractWords`、`genSentences` / `rewriteArticle` / `genQuestions` / `translate` (LLM)、`genAudio` (edge-tts 在 Cloud Run 跑，附逐字時間點)
- `cloud_firestore` (`persistenceEnabled: true`)
- 上傳：`card_progress` 每次寫入後同步寫一筆到 Firestore；離線時由 Firestore SDK 排隊
- 下載：登入或啟動時拉 `updatedAt > lastSyncAt` 的文件合併進 Drift，衝突取 `updatedAt` 較新者
- `review_log` 只上傳不下載合併 (append-only，重複無害)
- 匿名帳號認領：另一裝置輸入六位代碼 → Cloud Function 驗證後回傳 custom token → 登入同一 uid

6. 語音 (P2)

- 內建 mp3 (教材包附)：just_audio 播放，支援 setClip 播區段 (回音練習)、setSpeed 語速；離線快取常用檔 (PWA / 原生檔案快取)
- 退回方案 flutter_tts：Web 走 Web Speech，原生走系統 TTS
- 錄音 record：Web 產 webm，原生產 m4a；只存記憶體，不落地
- 自己的音檔：原生存 app 文件夾，Web 存 IndexedDB；沒有時間點，不反白、不能回音練習
- 產生腳本沿用 tools/gen_audio.py (edge-tts，四國口音，整篇附逐字時間點)

7. 測試策略

層	工具	重點
srs_engine	test	每個函式的邊界 (box 0/top、ease 上下限、lapses 只在 reps>1 累加)、六個方案的 applyMark 對照、時區 (本地日期)、v1 對照案例 (v1 的 tests/scheduler.test.js 24 案例先移植) (從 v1 抽 30 個字的作答序列，驗證 v2 算出相同 box/ease/due)
data	drift in-memory、fake_cloud_firestore	匯入 v1 備份、合併規則、離線→上線
domain	test	use case 組合
presentation	flutter_test widget test	今日練習的三段流程、鍵盤快捷鍵
教材	tools/validate_course.dart	id 唯一、必填欄位、例句含目標字

CI：GitHub Actions，flutter analyze (零警告)、dart test、flutter test、教材驗證、Firestore 規則測試 (emulator)，通過才 build web。

8. 部署

```
flutter build web --release --base-href /vocab-trainer/ --pwa-strategy offline-first
```

- Actions 把 app/build/web 推到 gh-pages 分支，Pages 來源改成該分支
- legacy/index.html 在 v2 上線後保留 30 天，頁首加「新版在這裡」連結
- PWA：manifest.json 設好名稱與圖示，手機可加到主畫面

9. 里程碑對應

里程碑	交付	驗收
M1 引擎	packages/srs_engine + 測試 + docs/engine.md	dart test 全綠；v1 對照案例通過
M2 App 骨架	profiles / courses / practice / storage / backup	匯入 v1 備份後可練習，重開 App 進度仍在
M3 上線	Actions 部署、legacy 導向	手機開網址 5 秒內可操作
M4 同步	sync feature、Firestore 規則	兩台裝置 30 秒內一致

里程碑	交付	驗收
M5 延伸	reading（先聽再看、逐字反白、逐句中譯、回音練習） / audio / Android	Play 內測

10. ADR 索引

- [ADR-0001 排程引擎做成純 Dart 套件](#)
- [ADR-0002 後端用 Firebase 而非自建 API](#)
- [ADR-0003 不做 LIFF 入口](#)
- [ADR-0004 教材做成 JSON 資料包](#)

第四章 執行計畫

原則：P0 做完就上線，不等 P1/P2。每個階段有明確的「完成定義」，做不完就縮功能，不延時程。本專案只用 stock_ledger 主線以外的時間。

開工前要先決定的事

- [] Firebase 專案：新開一個（帳號規則與 stock_ledger 不同）
- [] LINE Login channel、Apple（公司帳號下建 App ID / Services ID / Key）、Google OAuth client（流程見 07）
- [] Firebase Cloud Messaging：APNs Key、VAPID key
- [x] 不強制登入（App Store 5.1.1）：訪客完整可用，登入併入進度
- [x] v2.1 的 AI 模型：Claude Opus 5，費用由平台付（見 06）；v2.0 代做用 Claude Code
- [] 第一批教材：公用 gsat（從 v1 轉）+ 班級講義 英檢中級 2-5~2-10（從 v1 轉成 lessons）
- [] v1 使用者的切換日：v2 上線後通知，並確認她已下載備份

階段 0：專案骨架（半天）

- [] flutter create app 與 dart create -t package srs_engine，melos 或 pubspec workspace 管理
- [] analysis_options 用 flutter_lints 加專案規則，零警告
- [] GitHub Actions：analyze + test，先不部署
- [] legacy/ 資料夾建立（先不搬 v1）

完成定義：melos run analyze 與 melos run test 在 CI 綠燈。

階段 1：srs_engine（M1，W2）

- [] models：CardProgress、Mark、QuizMode、CardKind
- [] ladder.dart：levelOf、nextGap
- [] grader.dart：applyMark，含 ease 上下限、box 下限、lapses 條件
- [] mode.dart：quizType
- [] queue.dart：buildQueue、requeueIndex
- [] 測試：邊界案例 + v1 對照案例（用 v1 備份檔抽 30 個字的 log 回放）
- [] docs/engine.md：每個參數的來源與理由（從 README 設計決策展開）

完成定義：測試全綠、覆蓋率 100%、對照案例零差異。

階段 2：App P0（M2，W3）

- [] features/auth：訪客模式、三種登入、登入時併入本機進度、角色選擇、路由守衛；Function lineAuth

- [] features/onboarding：三張啟動卡、coach mark 元件與 seen 記錄、老師端導覽
- [] core/layout：依視窗寬度與 displayFeatures 的版型切換，狀態保留（Riverpod 狀態不隨版型重建）；直橫式與摺疊機測試矩陣
- [] features/classes：建班、邀請碼、加入（Function `joinClass`）、成員；Firestore 規則 + emulator 測試
- [] core/storage：Drift schema、web/native 分岔、migration；core/sync：登入即同步、每張卡 LWW、離線佇列
- [] 帳號刪除（含強制備份）
- [] features/courses：目錄載入、教材 JSON 解析、`tools/cards2json.dart` 把 v1 的 W/EXTRA/META/MORPH/ARTS/ARTZH/SRC/LESSON_ARTS 轉成 `data/courses/gsat.json`（含拆字、逐句中譯、音檔路徑、講義文章對應）
- [] features/practice：三段式、四種作答、干擾項、鍵盤、○△× 建議
- [] features/backup：匯出 v2、匯入 v1/v2、合併規則
- [] features/settings：P0 需要的四項（goal、revCap、mode、字體）

完成定義：匯入 v1 備份後能完整練一場，關掉重開進度仍在；`flutter analyze` 零警告。

階段 3：上線（M3，W4）

- [] Actions 加 build web + 推 gh-pages
- [] Pages 來源切到 gh-pages；v1 搬到 `legacy/index.html` 並加導向
- [] PWA manifest、圖示
- [] 手機實測：Safari iOS、Chrome Android，首次載入時間記錄進 README
- [] 通知 v1 使用者切換，確認匯入成功

完成定義：新網址可用、v1 使用者進度無損、README 更新為 v2。

階段 4：老師端與 P1（M4，W8 後）

- [] features/teacher：儀表板（彙總查詢、要注意規則）、班級管理、教材庫
- [] 上稿三步介面（v2.0 由開發者操作：抽字、審核、文章、產音檔、發布）
- [] features/library、features/calendar（三尺寸月曆格）
- [] core/notify：FCM / Web Push 註冊、Function 排程提醒
- [] 教材版本檢查與背景更新
- [] 設定其餘項目

完成定義：兩台裝置 30 秒內一致；離線 50 題後恢復連線自動同步。

階段 4a：任務系統（M4 內）

- [] 資料：assignment_templates、assignments（週、班級 / 學生、任務列）、task_logs（日期、證據）
- [] 老師端：範本編輯、套用到週、指派、本週表格子、聽證據

- [] 學生端：今日任務清單、本週表、錄音上傳 (Storage)、課外閱讀紀錄
- [] 儀表板：本週完成率、連續未完成、聽寫正確率

階段 4b：v2.1 (M5 之後)

- [] AI 上稿 Functions：genSentences、rewriteArticle、genQuestions、translate；老師端「AI 產」按鈕
- [] LINE 提醒 (Messaging API)
- [] 老師自助上稿開放

階段 5：延伸 (M5，視主線進度)

- [] features/reading：先聽再看、逐字反白、逐句中譯、隱性 wpm、重讀排程、上課前預習
- [] core/audio：just_audio 播內建 mp3 (區段播放)、回音練習狀態機、逐句跟讀、錄音、自己的音檔
- [] features/scan：找出生字 (內建字典 data/dict.json、線上查、字庫字排入下一批)
- [] features/lessons：字組上課日、優先排程、過一遍
- [] 列印今日
- [] Android build、Play 內測
- [] 第二、三套教材包

每階段結束要做的事

1. README 的路線圖勾選
2. docs/ 若有決策改變，補一份 ADR
3. Notion 開發筆記記錄踩到的坑 (去識別化)
4. 更新 ~/914Apps/README.md 的專案狀態

與 stock_ledger 的關係

兩個專案共用同一套結構與心智模型。srs_engine 是 portfolio_engine 的縮小版練習：先在這裡把「純 Dart 套件、測試先行、App 只做組裝」跑順，回頭做 stock_ledger 的計算引擎會更快。若兩者時間衝突，stock_ledger 優先。

第五章 決定紀錄

2026-09-11（回覆線框第 18 頁）

項目	決定	落地位置
不登入試用	允許。訪客可完整使用，加入班級 / 同步 / 老師身分才登入	規格書 §0、線框 p2/3/6
訪客提示	今日卡片的「加入班級 / 登入同步」只顯示前三天，之後收進設定	規格書 §0
登入方式	Google、LINE、Apple；申請流程見 07	07_帳號申請流程
iOS 開發者帳號	用公司既有帳號，個人帳號先不申請	07 §B
AI 上稿	v2.0 由開發者代做（Claude Code + Pro 訂閱）；v2.1 接 Claude API，Pro 訂閱不能直接串	06_AI上稿模型評估
每個字的 ○△×	老師儀表板要看到，但 不是使用者最後一次點的標記，而是由累積紀錄推導 （規則見規格書 §3a）	規格書 §3a、線框 p14
同時開兩套教材	可以，每日額度共用，今日清單先講義後公用	規格書 §3
目標完成日	全域一個（例如考試日）；沒設定就是「程度提升」模式，試算不顯示學完日反推	規格書 §8
回音練習	三輪後停下等按	已是現況
行事曆	一律月曆格（手機也是）；視覺稿之後用 Claude Design 出	線框 p11
本機提醒通知	要：FCM（Android / iOS）+ Web Push；預設開，可關	規格書 §10、07 §D
教材更新	App 啟動時檢查伺服器上的教材版本，自動更新	規格書 §4、技術文件 §4
老師儀表板	欄位維持，加「每個字的 ○△×」（推導值）	線框 p14

2026-09-11（補 2）

項目	決定	落地位置
首次使用引導	三張啟動卡可跳過 + 情境 coach mark 各一次 + 老師端三步導覽	規格書 §0a、線框 p7
直橫式與摺疊機	直式為主、橫式可用；依視窗寬度重排不丟狀態；摺疊機避鉸鏈、封面 / 內螢幕 / 半開三態	規格書 §9d、線框 p6

2026-09-11（補）

項目	決定	落地位置
跨教材重複字	進度以「字」為單位跨教材共用，不重複複習；教材只引用並附例句；上稿時標出已存在的字	規格書 §3、§5、§7；技術文件 §3

2026-09-10

- 角色分流（學生 / 老師）、班級模型、老師儀表板與上稿、v2.0 / v2.1 分期
- 公開檔案不出現出版社、作者、講義原檔名、原文標題、商業品牌；出處放 private/
- 不做「只練這組」；講義文章在上課前一天排預習

第六章 AI 上稿模型評估

日期：2026-09-11 | 狀態：評估，v2.0 先由開發者代做，v2.1 才接 API

1. AI 在上稿裡做什麼

任務	輸入	輸出	每份講義的量
產例句與中譯	單字、詞性、中文、講義原句（若有）	一句例句 + 中譯，用法筆記	40—60 字
改寫文章	原文（僅班級可見）	同主題原創文章 180—220 字，去品牌	1—2 篇
出理解題	改寫後文章	3 題四選一 + 解析	1—2 篇
逐句中譯	改寫後文章	每句中譯（句數須與分句規則一致）	12—17 句 / 篇
拆字	單字	字首 / 字根 / 字尾 + 中文，不確定就留空	40—60 字

全部是「文字進、文字出」的單次呼叫，不需要工具呼叫或代理迴圈。輸出用結構化格式（JSON schema）拿，程式直接寫入教材。

2. 訂閱與 API 是兩件事

Claude Pro 訂閱不能直接串進 App。Pro / Max 是 claude.ai 與 Claude Code 的個人使用方案；App 後端要呼叫模型必須用 Claude API，在 Console（console.anthropic.com）另外開帳號、放儲值，按用量計費。兩者帳單分開。

這對分期的意義：

- v2.0（開發者代做）：就是現在的流程，您在 Claude Code 裡用 Pro 訂閱整理講義，不多花錢。
- v2.1（老師自助）：Cloud Function 用 API key 呼叫，按量付費。金鑰放在後端，永遠不進 App。

3. Claude 模型與價格（API 牌價，2026-06 快取）

模型	模型 ID	輸入 \$/百萬 token	輸出 \$/百萬 token	適合
Claude Opus 5	claude-opus-5	5.00	25.00	預設。例句品質、改寫的自然度、中譯的準確度都最穩
Claude Sonnet 5	claude-sonnet-5	2.00	10.00	省一半以上，例句與中譯品質仍好；改寫文章略平
Claude Haiku 4.5	claude-haiku-4-5	1.00	5.00	拆字、簡單分類這類小任務

快取讀取另有折扣；批次 API 半價但最長 24 小時才回，不適合老師等著看結果的上稿情境。

4. 一份講義的費用估算

以 2-10 這種規模（60 字、2 篇文章）估：

項目	輸入 token	輸出 token
例句 + 中譯 + 筆記 60 字（分 3 批）	約 6,000	約 5,000
改寫 2 篇 + 題目 + 逐句中譯	約 3,000	約 3,500
拆字 60 字	約 1,500	約 1,500
合計	約 10,500	約 10,000

模型	每份講義	換算台幣（以 1:32）	每週一份、一年
Opus 5	約 \$0.30	約 10 元	約 500 元
Sonnet 5	約 \$0.12	約 4 元	約 200 元
混用（改寫用 Opus、其餘 Sonnet）	約 \$0.18	約 6 元	約 300 元

音檔不用 AI，走現有的免費語音套件在伺服器產，費用是 Cloud Run 的執行時間，一份講義幾秒。

結論：費用不是決策因素，一年不到一頓飯。決策因素是品質與由誰付。

5. 建議

1. 預設 Opus 5，全部任務用同一個模型。理由：講義內容會被學生反覆看、被老師署名發布，品質差異比每年 300 元重要；單一模型也讓快取與提示維護簡單。
2. 費用由平台付（也就是您的 Console 帳戶），老師端不用設定金鑰。老師人數少、用量可預估；等有多位老師再考慮按班級計費。
3. 多方選取不需要現在做。程式上把「呼叫模型」包成一個介面（輸入任務與資料、輸出 JSON），之後要換或加供應商只改那一層；但第一版只接一家，避免同時維護兩套提示與評估。其他供應商的價格請以各自官網為準，本文件不代為報價。
4. 每個任務都有可驗證的輸出檢查：例句必須包含該字或其變化形、逐句中譯句數必須等於分句數、題目答案必須在選項內、改寫文章不得出現原文連續 8 字以上、品牌與人名黑名單。不過檢查的結果退回重產一次，再失敗就標給老師看。
5. 老師永遠是最後一關：AI 產的內容以草稿呈現，老師可改，發布前要按確認。

6. 技術做法（v2.1 時實作）

- Cloud Functions（Node / TypeScript）用官方 `@anthropic-ai/sdk`，模型 `claude-opus-5`，`thinking: {type: "adaptive"}`，長輸出用串流。
- 結構化輸出：`output_config.format` 給 JSON schema，程式不用解析自由文字。
- 提示快取：系統提示（格式規則、黑名單、範例）固定放前面，講義內容放後面，同一份講義的多次呼叫共用快取。
- 每次呼叫記錄 `usage`，儀表板可看每份講義的實際花費。
- 金鑰放 Secret Manager；Function 只接受已登入且 role 為 teacher 的呼叫。

7. 要決定的事

- ☐ 費用由平台付（建議）還是老師自備金鑰？
- ☐ 改寫文章要不要限制長度與程度（例如英檢中級 180—220 字、CEFR B1）？建議要，放在提示裡。

第七章 第三方登入與服務申請

日期：2026-09-11。三者都在 v2 階段 0 前完成，因為登入是 P0。

A. LINE Login channel

用您現有的 LINE Developers 帳號即可，和 line_bento_order 的 Messaging API channel 可以放在同一個 Provider 下。

1. 到 developers.line.biz 登入，進入 Provider（沒有就建一個，名稱用品牌名，例如「914Apps」）。
2. Create a new channel → **LINE Login**。填：Channel name（單字練習台）、description、App types 勾 **Web app** 與 **Mobile app**、Email address。
3. 建好後在 Basic settings 記下 **Channel ID** 與 **Channel secret**（放 Firebase Functions 的 Secret，不進 App）。
4. LINE Login 分頁：
5. Callback URL 填 Web 的回呼網址，開發期用 `http://localhost:xxxx/auth/line`，正式用 `https://<你的網域>/auth/line`。
6. iOS bundle ID、Android package name 與 package signature（SHA-1 轉的）填進去，行動端 SDK 才能用。
7. 如果要拿學生的 Email（選用），在 Basic settings 申請 **OpenID Connect email permission**，要上傳隱私權政策網址，審核幾天。不拿 Email 也能運作，只用 LINE userId 當身分。
8. Channel 預設是 Developing 狀態，只有管理員與測試者能登入；正式上線前在 Basic settings 按 **Publish**。

程式端：App 用 `flutter_line_sdk` 取得 access token → Cloud Function 呼叫

`https://api.line.me/oauth2/v2.1/verify` 驗證 token 與 channel ID →

`admin.auth().createCustomToken(lineUserId)` → App 用 custom token 登入 Firebase。Web 端走 LINE 的網頁授權流程，回呼後同一支 Function。

B. Apple

公司已有 Apple Developer Program 帳號，不用另外申請，但要在該帳號下做四件事（需要帳號的 Admin 或 App Manager 權限）：

1. **App ID**：Certificates, Identifiers & Profiles → Identifiers → 新增 App ID（Bundle ID 例如 `com.914apps.vocabtrainer`），Capabilities 勾 **Sign in with Apple** 與 **Push Notifications**。
2. **Services ID**（給 Web 用）：Identifiers → 新增 Services ID，勾 Sign in with Apple，設定 Domains（你的網域）與 Return URLs（Firebase 的回呼 `https://<project>.firebaseapp.com/__/auth/handler`）。
3. **Key**：Keys → 新增，勾 Sign in with Apple，選上面的 App ID，下載 `.p8`（只能下載一次），記下 Key ID 與 Team ID。
4. **Firebase**：Authentication → Sign-in method → Apple → 啟用，填 Services ID、Team ID、Key ID、`.p8` 內容。

另外：

- App Store 上架時 App Privacy 要填資料收集項目 (Email / 名稱 / 使用資料)，與 5.1.1 相關的「強制登入」已在設計裡。
- 推播用 **APNs Key** (Keys → 勾 Apple Push Notifications service)，上傳到 Firebase Cloud Messaging。
- 公司帳號發布的 App，開發者名稱會顯示公司名；若希望顯示個人或品牌，要另開帳號，這是您說先不申請的那個。

C. Google

1. Firebase Console 建專案 (新開一個，不與 stock_ledger 共用)。
2. Authentication → Sign-in method → Google → 啟用；填支援 Email。
3. Android：在 Firebase 專案加 Android App，填 package name 與 **SHA-1 / SHA-256** (debug 與 release 各一組)，下載 `google-services.json` (不進 git，用 CI 密鑰注入)。
4. iOS：加 iOS App，填 bundle ID，下載 `GoogleService-Info.plist` (同上不進 git)。
5. Web：加 Web App，取得 config；OAuth 同意畫面在 Google Cloud Console 設定，App 名稱、支援 Email、隱私權政策網址。

D. 通知 (FCM / Web Push)

1. Firebase → Cloud Messaging：Android 自動可用；iOS 上傳 APNs Key；Web 產生 **VAPID key pair**。
2. App 端 `firebase_messaging`；Web 需要 `firebase-messaging-sw.js` service worker。
3. 提醒內容由 Cloud Function 排程 (每天讀書日的固定時間) 依每位學生的到期數送出，例如「今天有 23 個字到期，新字 8 個」。使用者可在設定關閉。

E. 隱私權政策與服務條款

三家登入與兩個商店都會要求公開的隱私權政策網址。放在 GitHub Pages 的 `/privacy` 即可，內容：收集什麼 (登入身分、暱稱、練習進度)、用途、保存、刪除方式 (App 內刪除帳號)、第三方服務 (Firebase、LINE、Google、Apple)。這份我可以先擬草稿。

順序建議

Firebase 專案 → Google 登入 (最快，先讓開發能跑) → LINE Login channel (審核 Email 權限要等) → Apple (上架前完成即可，但 Services ID 要在測 Web 登入前設好) → FCM。

附錄 v1.7 使用說明（現行網頁版）

網址：<https://joyce914apps.github.io/vocab-trainer/> 手機、平板、電腦的瀏覽器都能開，不用安裝。進度存在這台裝置的這個瀏覽器裡，換裝置請看「備份」。

一、每天怎麼用（5 分鐘看懂）

1. 打開網頁，停在「今日」分頁。最上面的卡片告訴你今天要做什麼：新字幾個、複習幾個、有沒有文章、有沒有講義要預習。
2. 往下就是練習卡，照著做：
3. ① **建立印象**：第一次見到的字。看例句、聽發音、看拆字，把字手打一次，按「記住了，考我」。
4. ② **回想**：例句挖空，預設是四選一；要會拼寫的字到第四階起會出拼寫題。
5. ③ **標記**：程式會建議 ○△×，你可以改。○ 會了、△ 有卡點、× 不會。你比程式清楚剛剛是真會還是矇到。
6. 答錯的字幾題後會再出現一次，答對的新字當場最後再看一次。這是設計，不是重複。
7. 做到卡片說「今天的業績達成」就可以停。沒做完也沒關係，明天到期的字會自己排進來。

鍵盤：Enter 送出 / 下一步，1-4 選選項，○△× 對應 1、2、3。

二、五個分頁

分頁	用途
今日	每天的練習。頂端卡片是今天的清單與業績
字庫	所有字的熟悉度。可搜尋、篩選（生疏、退步過、今天到期、未學、已畢業、依字根 / 字尾分組、某個字組）
文章閱讀	先聽再看的閱讀與跟讀（見第四節）
複習行事曆	未來 30 天每天要複習幾個字、哪幾天休息、哪天有文章與講義預習
設定與備份	字組開關、每日新字、讀書日、方案、聲音、備份、重置（見第五節）

另外「貼上字卡」分頁用來自己加字（見第六節），「教材來源」列出每份講義的字數與進度。

三、熟悉度與複習節奏

- 每個字有自己的「階」。答對升一階，下次出現的間隔變長；答錯退階，很快再出現。
- 預設節奏：新字前四天每天出現，之後隔 7、15、30、60 天。前四天是遺忘最快的區段，所以不順延、休息日也會露臉（只做這些字，約 5 分鐘）。
- 「已畢業」的字仍會每 60 天回來一次。
- 字庫每個字下面有小方塊，越多綠色越熟。

四、文章閱讀

1. 點一篇文章。先**只有播放器**，看不到文字。按播放，聽完一遍再按「聽完第 1 遍了，看文章對照」。想再聽幾遍都可以。
2. 看文章時，唸到的字會像螢光筆一樣**逐字反白**。看不懂的句子點一下出現中譯，再點收起。
3. 練發音有三種：
4. **回音練習**（建議）：4~6 個字一段。播 → 停，先別出聲，聽心裡留下的回音 → 照著回音唸 → 再播對照。同一段走三輪，溜了按「下一段」。每天十分鐘，時間到會自動結束，明天從那一段接著。
5. **逐句跟讀**：整句，聽 → 停 → 唸 → 再聽。
6. **錄音 / 播我的**：錄自己唸的，和原音比。
7. 讀完按「讀完了，看理解題」。答完題才能按「全部中譯」。閱讀速度（wpm）會自動算，不用按計時；第一次偏低是正常的，重讀那天的數字才是速度。
8. 每篇會排 4 天後、14 天後重讀。

自己的音檔：有老師的錄音可以按「用自己的音檔」上傳，只存在這台裝置，會優先播放，但沒有逐字反白與回音練習。

五、設定

- **字組**：勾選要練哪些。每週講義各是一組，旁邊填上課日，這組還沒學的字就會排到新字最前面，並顯示「還有 N 字、每天約 X 個」；講義的文章會在上課前一天出現「上課前預習」。**不要為了趕講義關掉其他字組**，複習會堆起來。按「過一遍」可以到字庫依講義順序看整組。
- **每日新字**：3—60。**讀書日**：勾一週哪幾天讀。**目標完成日**：填了會反推每日新字與建議方案。下方的「學習計畫試算」會即時算出學完日、畢業預估、每日負擔。
- **排程方案**：預設「考前混合」。另有固定 16 / 10 / 6—8 遍、經典 5 遍（第 2、4、7、15、30 天）、衝刺 8 週 / 4 週。換方案不會清進度。
- **作答方式**：自動（預設） / 選擇 / 拼寫。學測只有中譯英與作文需要拼得出來，所以預設選擇題。
- **拼寫題用聲音出題**：打開後拼寫題不給中文與挖空句，只播例句，聽到就打出目標字（聽寫）。準備聽力測驗時再開，學測衝刺期關著。
- **聲音**：口音（美、英、澳、加，多益英聽四種）、語速、進新字自動唸。單字、例句、文章都是內建的神經網路語音，離線才退回瀏覽器語音。
- **A— A+**：整體字級。**深色**：切換主題。

六、加字

- **貼文章找生字**：把文章貼進「貼上字卡」按「找出生字」。字庫已有的字會標熟悉度，勾未學的按「排入下一批」；新字自動帶入中文，字典沒有的按「線上查」，例句用文章那一句，也可「翻例句」。
- **貼上字卡**：一行一張，單字 | 中文 | 詞性 | 例句 | 例句中譯 | 用法筆記 | 音節 | P或R，來源名稱就是字組名稱。
- 自己加的字沒有內建音檔，會用瀏覽器語音。

七、備份與換裝置

「設定與備份」→「下載備份檔」存成 JSON，到新裝置按「從檔案還原」。建議每週下載一次。手機不方便存檔可用「產生備份文字」複製。清除進度或完全重置前，先備份；那兩顆按鈕要按兩次才會執行。

八、常見問題

- **休息日打開沒東西？** 正常。到期的複習順延到下一個讀書日；只有前四天的新字會露臉，用選擇題幾分鐘做完。
- **發音怪？** 設定裡換口音；手機裝了系統的高品質語音套件會更自然。內建 mp3 需要連線下載一次，之後有快取。
- **今天做不完？** 沒做的明天自動排進來，不用補。連續天數跨休息日不會斷。
- **字卡上的「要會拼寫 / 看得懂就好」可以改嗎？** 點卡片上的徽章即可切換；字庫也能整組切換。